

1. Analisi

1.1 Descrizione e requisiti

Il software realizzato si pone l'obiettivo di riprodurre una versione dettagliata del gioco da tavolo "Gioco dello Scudetto", ed è stato pensato per essere giocato con i propri amici grazie ad una gestione automatizzata dei turni.

L'applicazione è usufruibile da un massimo di 4 giocatori i quali si scontrano in incontri 1vs1 in un campo composto da 32 caselle con cui riprodurre situazioni tipiche di una partita di calcio come **calci di rigore** o **calci d'angolo**.

Tramite il lancio dei dadi è possibile proseguire nel campo da gioco, eseguendo l'azione specifica della casella in cui si è finiti. Nella prima parte della partita (le prime 16 caselle) il risultato ottenuto dal lancio dei dadi corrisponde alla somma di due dadi, mentre, raggiunta la casella di fine primo tempo, se ne usa solo uno. Non appena uno dei due player compie un giro completo di tabellone la partita termina, aggiornando opportunamente la classifica e procedendo con la prossima partita.

Requisiti funzionali

- L'applicazione permette di scegliere il numero di partecipanti (2-4), i quali sceglieranno un nome della squadra e un colore a essa associato. Non sarà possibile procedere se i nomi delle squadre sono uguali (o vuoti) e se il colore non è stato scelto
- L'applicazione crea un calendario con l'ordine delle partite che devono essere disputate e una classifica che verrà aggiornata partita dopo partita
- L'applicazione deve gestire opportunamente i turni e il lancio dei dadi
- L'applicazione deve gestire gli eventi di ogni casella, riproducendo l'effetto sul giocatore corretto
- L'applicazione deve gestire la fine della partita in corso, con conseguente aggiornamento della classifica e indicando, se ce ne sono ancora, la prossima partita oppure il vincitore dello scudetto con la classifica finale
- A questo punto è possibile eseguire un nuovo torneo con le stesse squadre (restart) o selezionare nuove squadre (menù)

Requisiti non funzionali

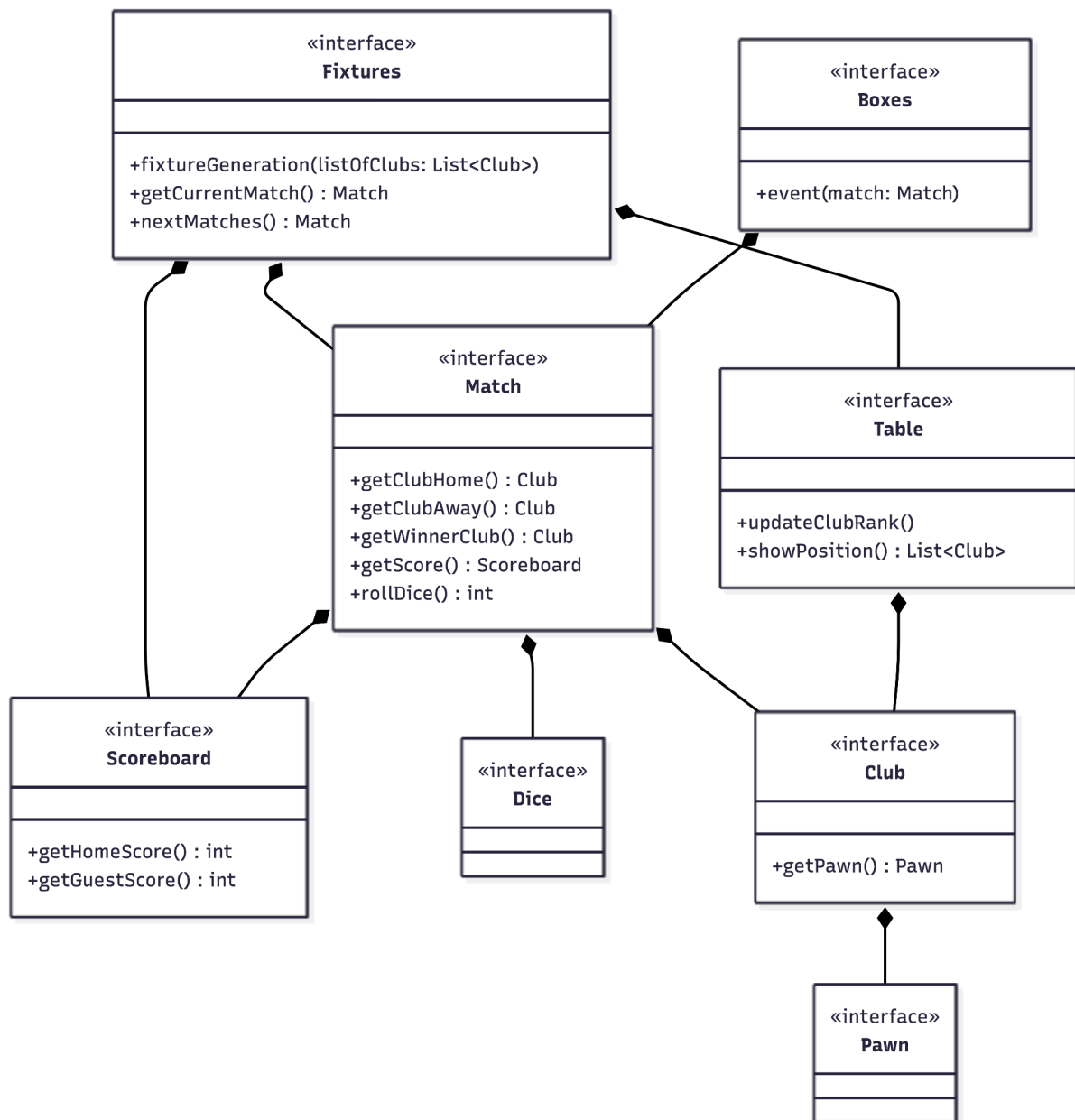
- **(Portabilità)** L'applicazione deve gestire correttamente diverse risoluzioni grafiche, adattando la dimensione delle componenti GUI, e deve funzionare sui sistemi Linux, Windows e MacOS

1.2 Modello del Dominio

Durante ogni partita i giocatori interagiscono con il tabellone tramite le proprie pedine che, dopo aver lanciato i dadi, eseguono le azioni previste dalla casella in cui si ritrovano.

I gol delle due squadre in partita vengono mostrati in una scoreboard durante la partita, in modo che l'esperienza di gioco sia più intuitiva e intrigante.

In seguito ad ogni partita i risultati ottenuti, come vittoria/sconfitta e differenze reti, contribuiscono all'aggiornamento di una classifica generale, decretandone alla fine il vincitore dello scudetto.



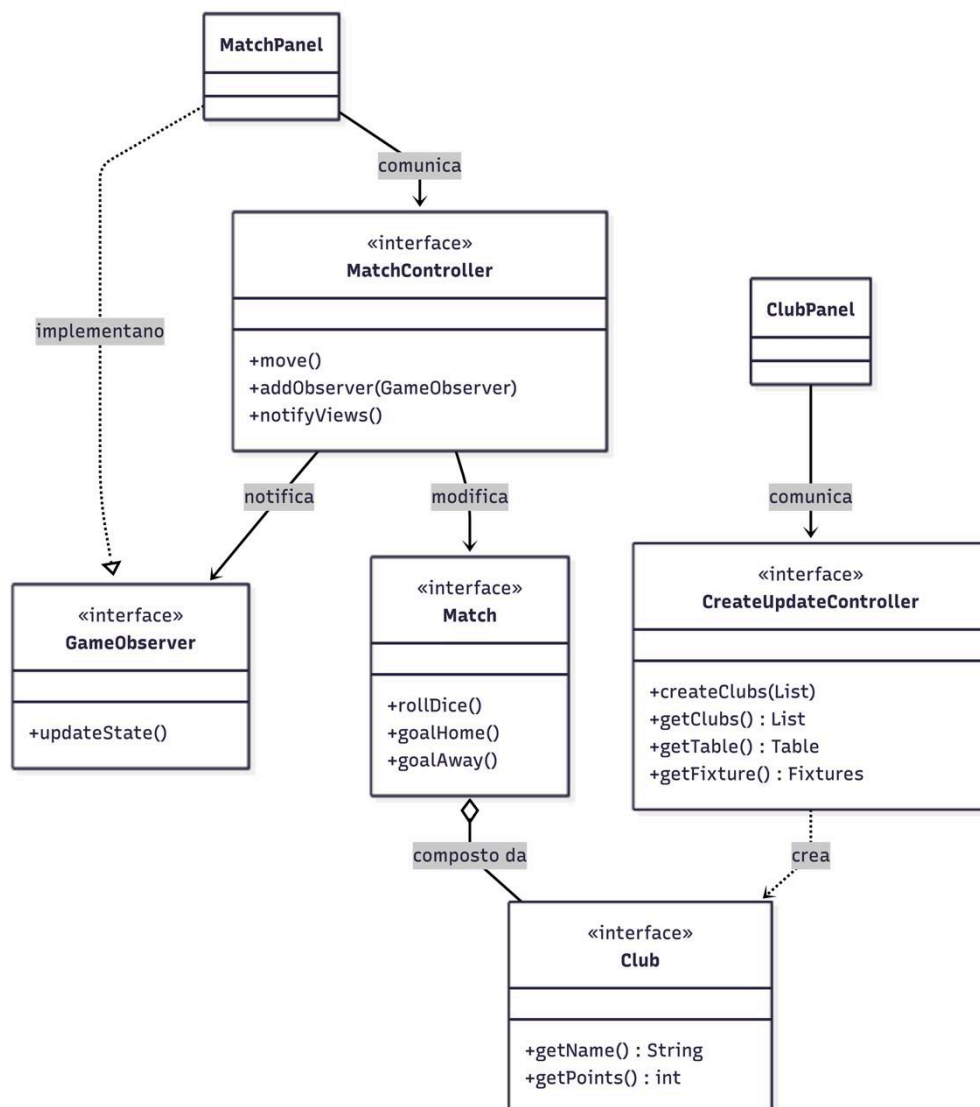
2. DESIGN

2.1 Architettura

L'architettura del Gioco dello Scudetto segue il pattern architetturale **Model-View-Controller(MVC)**. Tale scelta è stata adottata per garantire una chiara separazione delle responsabilità tra la logica di gioco e la rappresentazione grafica del gioco. Questa suddivisione favorisce la manutenibilità, l'estendibilità e il riutilizzo del codice.

A sostegno della separazione precisa dei compiti sono stati implementati tre controller distinti: MatchController, responsabile della logica di partita, CreateUpdateController che gestisce la creazione dei club e l'aggiornamento della classifica e Starter che coordina la navigazione tra le schermate.

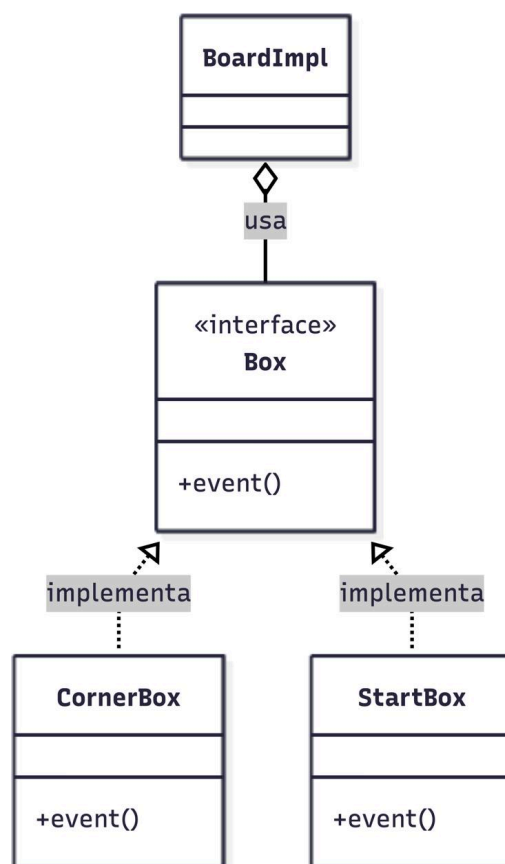
In questo modo il blocco grafico delle View è totalmente sostituibile, potendone quindi reinventare la grafica mantenendo un corretto funzionamento di quella logico/funzionale



2.2 Design Dettagliato

Tra le parti di design che sono state realizzate, ne presento due in particolare, le quali risultano fondamentali al corretto funzionamento della logica di gioco e di interscambio dei panel. Specifico che questo è stato realizzato con la collaborazione di tutti i componenti del gruppo.

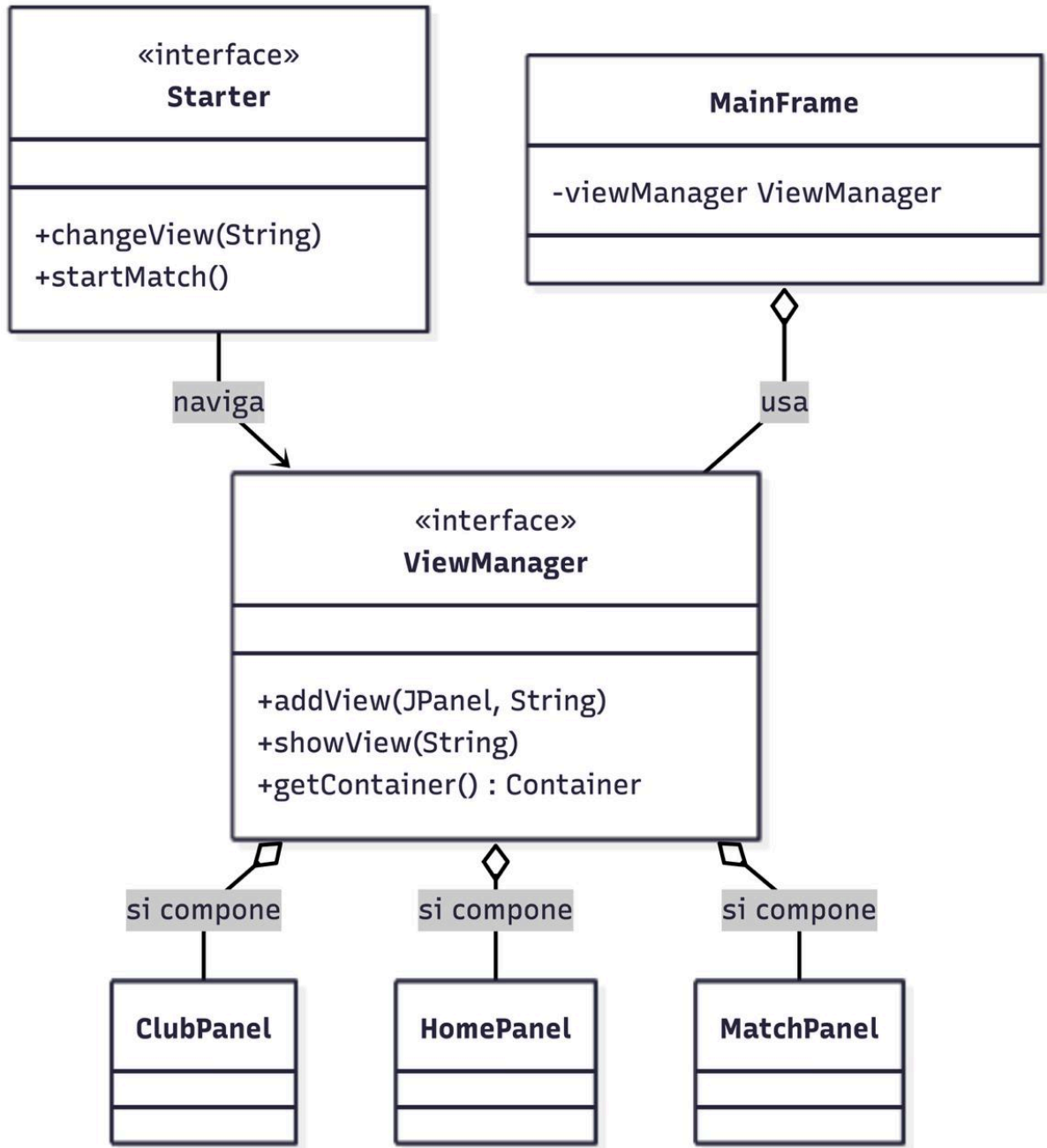
Per realizzare in maniera efficace la gestione degli eventi delle singole caselle è stato usato lo **Strategy Pattern**, in modo da specializzare l'evento di ciascuna.



Invece, per usufruire di un interscambio efficace dei Panel è stato pensato l'utilizzo di un **CardLayout**.

La view è composta da **MainFrame**, che rappresenta la finestra principale, e da un **ViewManager**, che gestisce la navigazione dei pannelli nel frame.

Alcuni pannelli implementano l'interfaccia **GameObserver** in modo da poter aggiornare il proprio stato sotto comando del controller. Per semplificare la visione non sono stati inseriti tutti i panel nell'UML.



2.2.1 Colinucci

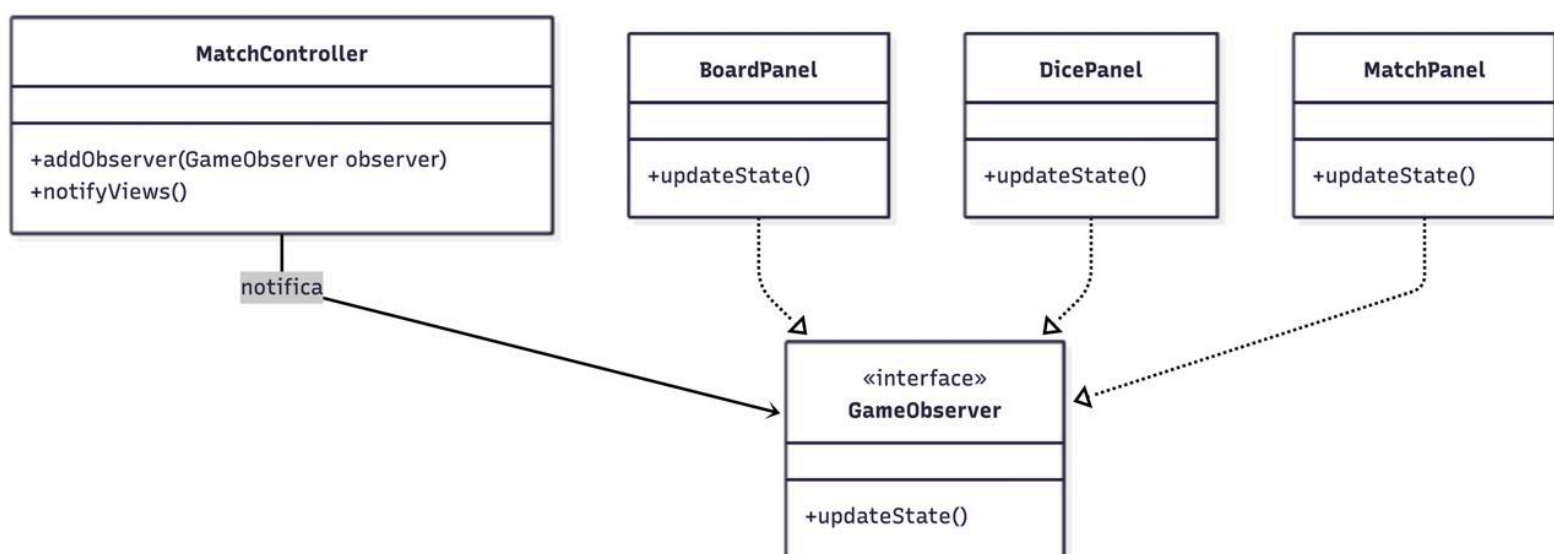
Gestione aggiornamenti view (Observer Pattern)

Problema

Durante lo sviluppo si è presentato il problema di come notificare le view di un cambiamento dello stato del gioco, senza che il controller mantenesse i riferimenti diretti ai vari pannelli grafici e quindi evitando di introdurre un forte accoppiamento tra i livelli di architettura e violando di fatto il principio di separazioni delle responsabilità.

Soluzione

Per risolvere il problema ho adottato il pattern Observer, implementando tramite l'interfaccia GameObserver che espone un unico metodo updateState(). Ogni pannello che necessita di aggiornarsi in risposta ad eventi di gioco implementa questa interfaccia e definisce il comportamento del metodo in base a cosa deve aggiornare. Il MatchController ha una lista di GameObserver e grazie al suo metodo notifyViews() è in grado di aggiornare tutte le implementazioni di GameObserver che possiede.



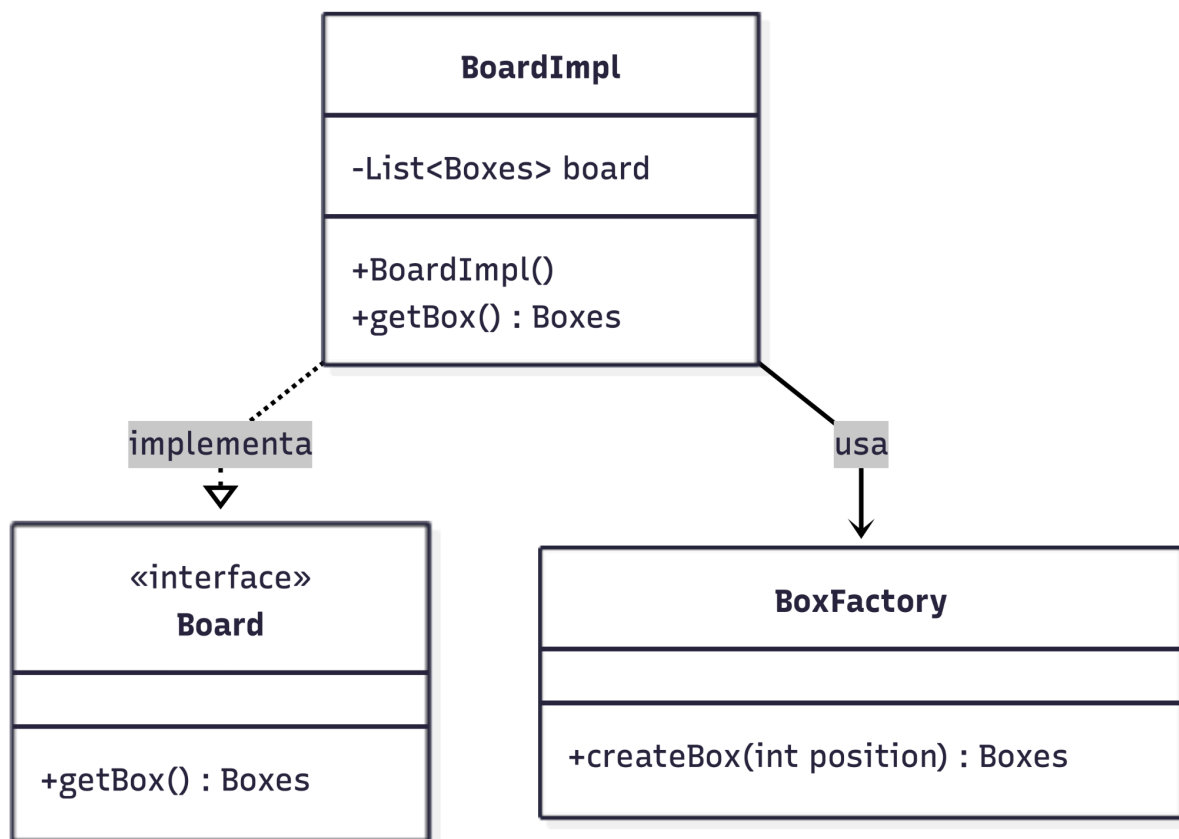
Gestione creazione delle caselle — Static Factory Method

Problema

Durante l'implementazione della classe BoardImpl si è presentato il problema di come creare le 33 Box del tabellone in modo pulito e manutenibile. La soluzione iniziale prevedeva 33 chiamate esplicite al costruttore direttamente dentro BoardImpl, una per ogni Box, rendendo il codice verboso, difficile da leggere e soggetto a errori in caso di modifiche alla disposizione delle caselle; risultava molto hardcoded.

Soluzione

Per risolvere il problema è stato adottato il pattern Static Factory Method, implementato nella classe BoxFactory che espone un unico metodo statico createBox(int position). Il metodo riceve la posizione sul tabellone e restituisce l'istanza della casella corretta tramite uno switch, raggruppando le posizioni che condividono lo stesso tipo di casella. Adesso il BoardImpl attua un ciclo dentro il suo costruttore e si fa creare la Box giusta per poi inserirla nella sua lista boxes.



2.2.2 Cecchini

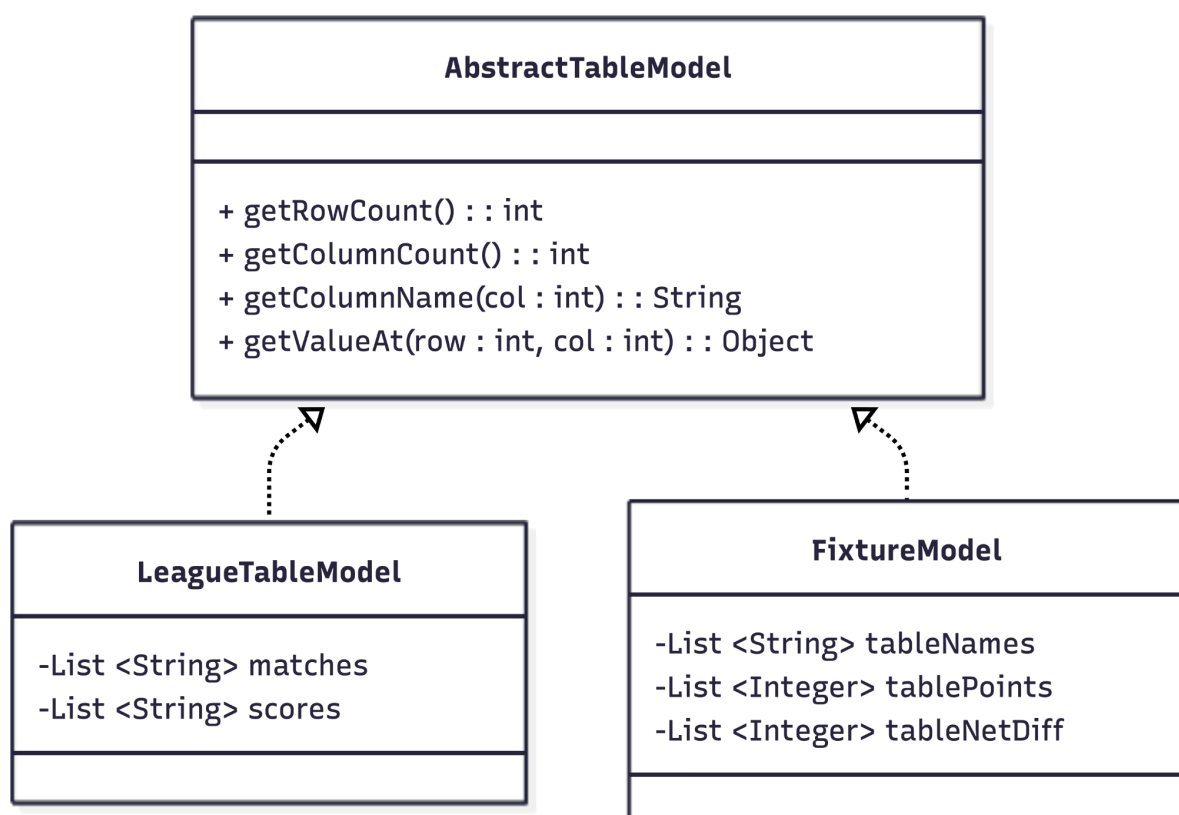
Visualizzazione dati con `TableModel` — Template Method

Problema

Per la realizzazione della visualizzazione della classifica e del calendario si è presentato il problema di come mostrare i dati del model in una `JTable`. La soluzione immediata sarebbe stata passare direttamente i dati alla tabella, ma `JTable` non è compatibile con strutture come `Map<Match, Scoreboard>` e accetta solo dati tabulari per righe e colonne. Inoltre, per la classifica, i dati da visualizzare (nome, punti, differenza reti) erano distribuiti nell'oggetto `Table` per l'ordine e nei singoli oggetti `Club`.

Soluzione

Per risolvere il problema sono state create le classi `FixtureModel` e `LeagueTableModel`, entrambe estensioni di `AbstractTableModel` di Swing. Questo approccio realizza il pattern Template Method: `AbstractTableModel` definisce lo scheletro del comportamento della tabella, mentre le due sottoclassi implementano i metodi astratti `getRowCount()`, `getColumnCount()` e `getValueAt()` fornendo i dati specifici. I dati vengono estratti dalla `Map<Match, Scoreboard>` e dagli oggetti `Club` e convertiti in liste parallele consultabili per riga e colonna. La `JTable` interagisce esclusivamente con il `TableModel`, senza conoscere la struttura dati sottostante.



2.2.3 Poli

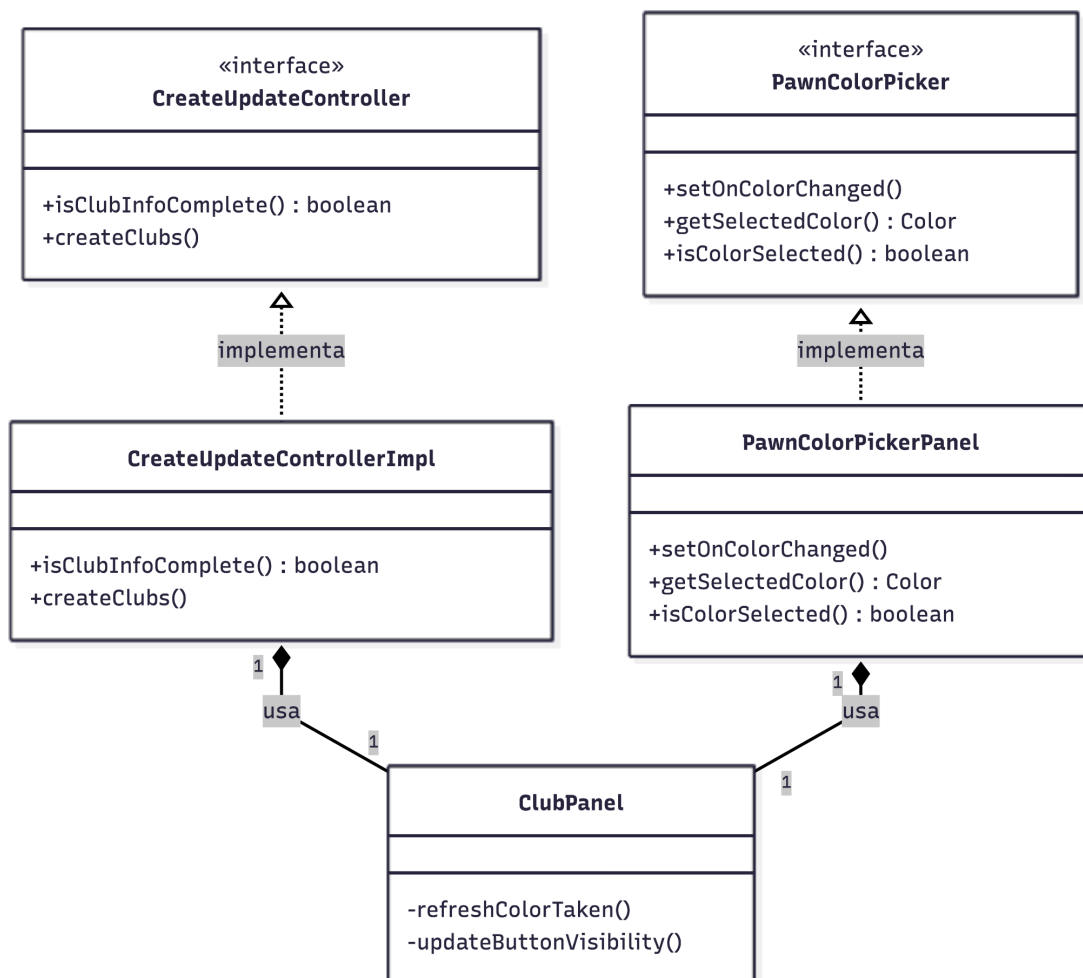
Gestione e Assegnamento dei colori ai Club

Problema

Durante lo sviluppo della classe ClubPanel si è presentato il problema di come permettere agli utenti di scegliere il colore della propria pedina in modo intuitivo ed efficace, riuscendo poi a memorizzarlo nel club in fase di creazione senza compromettere il pattern MVC

Soluzione

Il problema è stato risolto creando una classe dedicata alla sola gestione e rappresentazione dei colori da poter scegliere (PawnColorPickerPanel). All'interno di questa classe sono presenti metodi che permettono di aggiornare l'attuale stato dei colori, non permettendo la scelta del medesimo colore da parte di due o più team: in questo modo è stato risolto il problema della scelta del colore. Inoltre, sono presenti metodi volti alle operazioni di controllo della completezza delle informazioni dei club, protratte nel ClubPanel il quale richiama un metodo apposito del CreateUpdateController: isClubInfoComplete(). Infine per memorizzare il colore scelto da ciascuna squadra senza violare il pattern MVC, è stato usato il metodo getRGB() della classe Color, che memorizza un int, perfetto per essere memorizzato dalle classi del model.



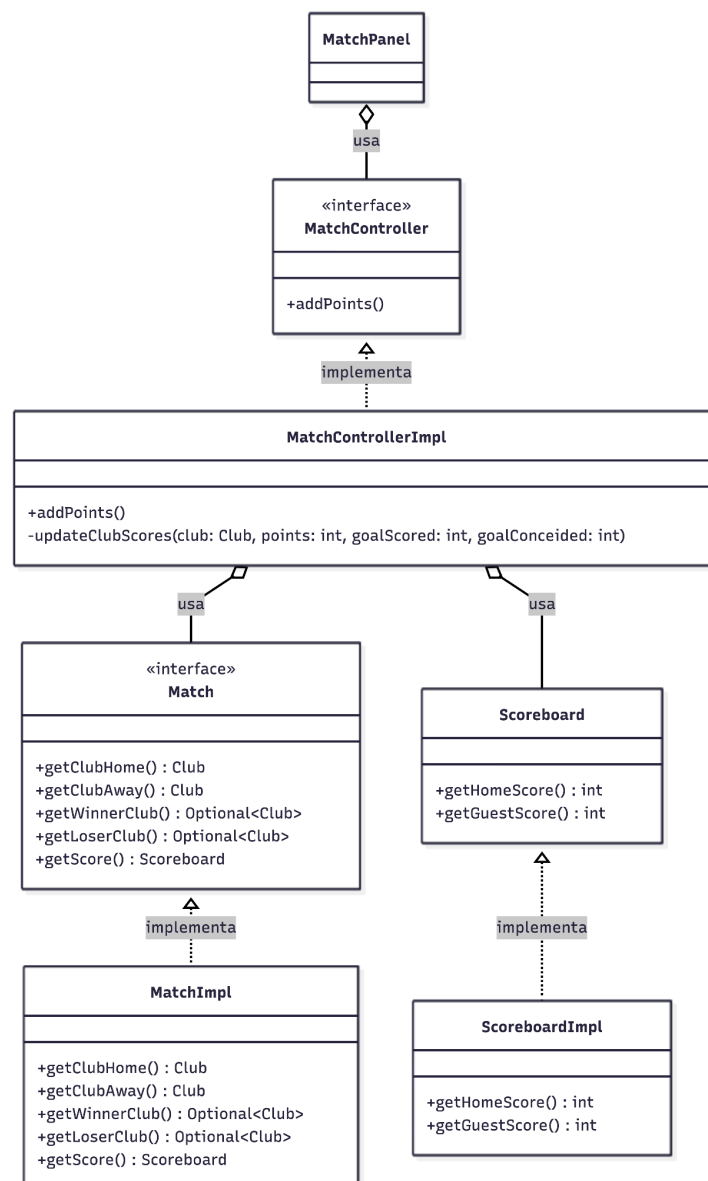
Aggiornamento della Classifica

Problema

Considerato che il modello generale del progetto è un “campionato” calcistico si vuole tener traccia di punti e differenza reti delle squadre in una classifica dedicata, in modo che dopo ogni partita si possa avere una panoramica generale dell’andamento di ogni squadra e alla fine poter decretare un vincitore.

Soluzione

Alla fine di ogni partita, tramite la collaborazione di due metodi del MatchController appositamente invocati(addPoints() e updateClubScores()), viene identificato un ipotetico vincitore (ipotetico in quanto la partita può finire anche in parità, di fatti implementato tramite Optional) in modo da poter poi assegnare correttamente i punti ad ambedue i Club.



2.2.4 Pasquinelli

Gestione dei turni

Problema

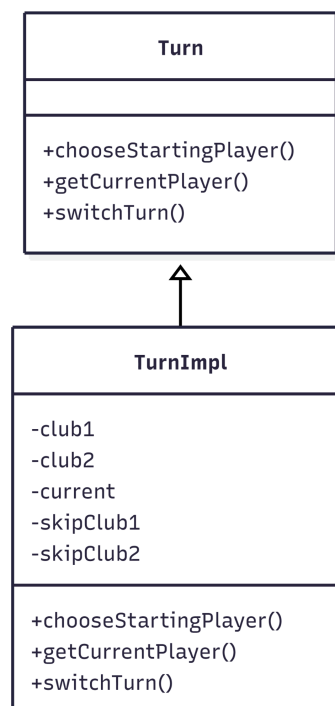
Durante lo sviluppo del gioco si è presentato il problema di gestire in modo centralizzato l'alternanza dei turni tra i due giocatori. Oltre al normale cambio turno, il regolamento del gioco prevede eventi speciali che modificano il normale svolgimento della partita, come ad esempio l'obbligo per un giocatore di saltare il turno successivo. Inserire questa logica direttamente nella classe Match avrebbe comportato una gran dipendenza tra la gestione della partita e la gestione dei turni, riducendo sia chiarezza che manutenibilità del codice.

Soluzione

Per risolvere il problema è stata introdotta l'interfaccia Turn e la sua implementazione TurnImpl, responsabile esclusivamente dei turni di gioco. La classe espone metodi per scegliere il giocatore iniziale, distinguere il giocatore corrente e passare il turno all'avversario.

In particolare la gestione del salto turno è stata centralizzata all'interno di TurnImpl tramite due flag associati ai giocatori. Quando una casella speciale richiede di saltare il turno successivo, essa si limita a notificare la classe TurnImpl attraverso il metodo setSkipTurn(). Agirà poi il meccanismo di cambio turno che applicherà il giusto comportamento.

Questa soluzione non implementa direttamente un design pattern specifico, ma applica il principio di Single Responsibility, concentrando tutta la logica relativa alla gestione dei turni in una singola componente dedicata. La soluzione adottata consente di separare la logica della partita dalla logica di gestione turni, migliorando la leggibilità del codice e rendendo più semplice l'aggiunta di nuove regole legate alla gestione dei turni.



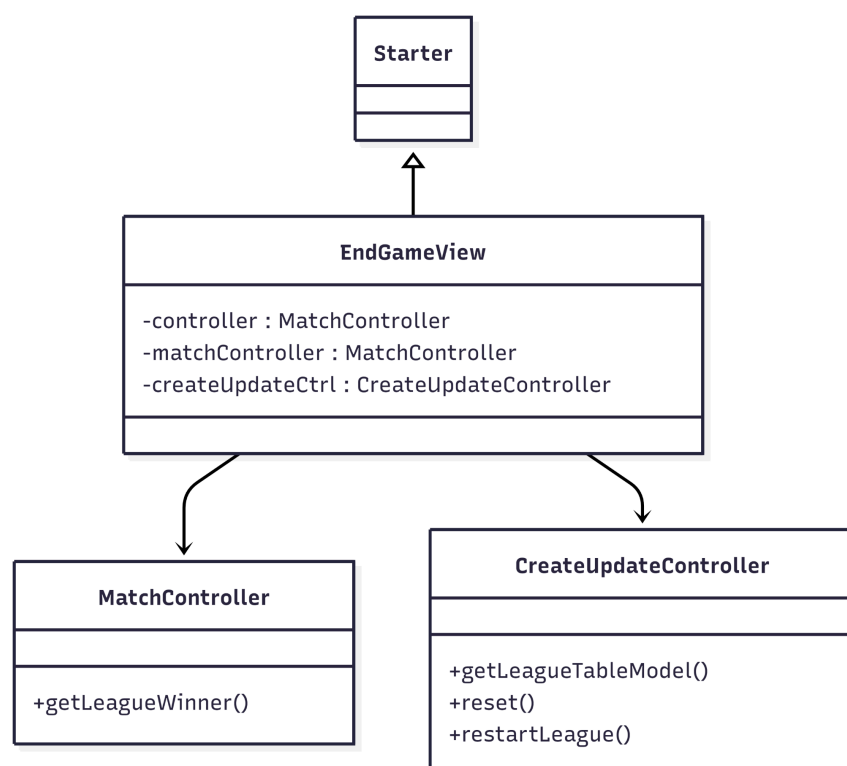
Visualizzazione dei risultati finali del campionato

Problema

Durante lo sviluppo del gioco si è presentato il problema di mostrare, al termine di tutte le partite del campionato, una schermata riassuntiva contenente il vincitore e la classifica finale con punti e differenza reti. Questa schermata doveva essere indipendente dalla logica di gioco e adattarsi a specifiche richieste.

Soluzione

Per risolvere il problema è stata realizzata la classe EndGameView, responsabile unicamente della presentazione grafica dei risultati finali. La vista ottiene dal controller le informazioni necessarie per costruire l'interfaccia, tra cui il vincitore e il modello della classifica. Facendo così la logica applicativa rimane separata dalla logica di presentazione e la view non deve conoscere i dettagli interni del modello. Grande attenzione è stata data all'adattabilità della schermata. Attraverso un ComponentListener la dimensione dei font e della tabella viene aggiornata automaticamente quando la finestra viene ridimensionata. Questa soluzione segue il pattern architetturale MVC dove la classe EndGameView rappresenta il componente view che comunica unicamente con il controller per ottenere i dati necessari. Tutto ciò per mantenere presentazione, logica applicativa e modello dati completamente separati.



3. Sviluppo

3.1 Testing Automatizzato

Per il testing automatizzato del progetto abbiamo utilizzato JUnit, in modo da verificare il corretto funzionamento dei componenti interni che costituiscono l'architettura portante dell'intero sistema. L'esecuzione dei test è autonoma e non richiede alcun tipo di intervento da parte dell'utente.

Ecco nello specifico i vari componenti che abbiamo verificato:

TestClub: realizzato al fine di garantire la corretta creazione e gestione dei dati di un club, testando la creazione, aggiunta di punti e differenza reti. Inoltre vengono verificati anche i casi limite in cui si modificano i campi di un club in modo scorretto e non coerente.

TestFixturees: controlla che il calendario venga creato correttamente, che ogni squadra giochi il giusto numero di volte, che il calendario venga eliminato correttamente e infine che sia possibile vedere la partita successiva senza influire nel corretto scorrimento delle partite.

TestGoalNet: utilizzato per verificare la dinamica del rigore, quindi si accerta che dove è presente il portiere il tiro viene parato e viceversa.

TestMatch: si concentra sul testare l'entità partita, accertandosi del meccanismo di aggiunta e rimozione goal, della selezione del vincitore e del perdente e infine del comportamento dei dadi nelle due metà campo.

TestPawn: si accerta del corretto comportamento della pedina. Verifica la creazione della pedina, della modifica della posizione e del colore che gli viene attribuito.

TestScoreBoard: realizzato per verificare il corretto funzionamento del punteggio di una partita. Si accerta che funzionino correttamente aumentare e diminuire i gol di una squadra, impostare i gol di una squadra ad un determinato valore e che il valore dei gol sia corretto sulla base delle operazioni effettuate precedentemente.

TestTable: implementato per accertare la correttezza della classifica. Controlla che la squadra in cima alla classifica sia quella con più punti, in caso di parità di punti si accerta che la prima squadra è quella con la differenza reti maggiore.

TestTurn: Sviluppato per testare la correttezza della classe turno, di conseguenza verifica la scelta del primo giocatore, il cambio di turno e il rimanere fermi per un turno.

BoxesTest: In questa collezione di test viene assicurato il funzionamento di ogni casella, in particolare vengono testati il nome, immagine, posizione e l'evento peculiare di ogni singola casella.

TestFixtureModel e TestLeagueTableModel: Realizzati al fine di testare la correttezza del fixture model e league table model, quindi i nomi delle colonne, numero di righe e il valore giusto restituito in base a colonna e riga.

TestCreateUpdateController e TestMatchController: Sviluppati per verificare la correttezza dei due controller più complessi e in particolare si assicurano il funzionamento dei metodi più delicati.

3.2 Note di sviluppo

3.2.1 Cecchini

1. Utilizzo di Lambda:

<https://github.com/NicolasColi05/Gioco-Scudetto/blob/0671560b68cbeb07837fbcdaebd9d81efbbe3451/src/main/java/giocoscudetto/view/impl/result/PreMatchView.java#L115>

<https://github.com/NicolasColi05/Gioco-Scudetto/blob/0671560b68cbeb07837fbcdaebd9d81efbbe3451/src/main/java/giocoscudetto/view/impl/result/PreMatchView.java#L120>

<https://github.com/NicolasColi05/Gioco-Scudetto/blob/0671560b68cbeb07837fbcdaebd9d81efbbe3451/src/main/java/giocoscudetto/controller/impl/CreateUpdateControllerImpl.java#L209>

2. Utilizzo di stream per l'iterazione su singole entità di club:

<https://github.com/NicolasColi05/Gioco-Scudetto/blob/0671560b68cbeb07837fbcdaebd9d81efbbe3451/src/main/java/giocoscudetto/controller/impl/CreateUpdateControllerImpl.java#L152-L202>

3.2.2 Colinucci

1. Utilizzo di Graphics2D per disegnare le varie pedine con la loro ombra e colore appropriato, e disegnare le caselle del tavolo da gioco:
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/4ddec604f7a868f0891ef260bec3e2ad1392bdef/src/main/java/giocoscudetto/view/impl/match/BoardPanel.java#L199>
2. Utilizzo di stream+Lambda expression in più punti(ne specifico uno):
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/4ddec604f7a868f0891ef260bec3e2ad1392bdef/src/main/java/giocoscudetto/view/impl/match/NetPanel.java#L141>
3. Utilizzo di javax.swing.Timer per simulare delle animazioni di tiro dei dadi nei vari eventi:
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/4ddec604f7a868f0891ef260bec3e2ad1392bdef/src/main/java/giocoscudetto/view/impl/match/EventPanel.java#L11>

3.2.3 Pasquinelli

1. Lambda per action listeners
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/28e0e55fc57b382eb0a174a4bd83014e2b9d773f/src/main/java/giocoscudetto/view/impl/result/EndGameView.java#L142>

<https://github.com/NicolasColi05/Gioco-Scudetto/blob/28e0e55fc57b382eb0a174a4bd83014e2b9d773f/src/main/java/giocoscudetto/view/impl/result/EndGameView.java#L148>

3.2.4 Poli

1. Utilizzo degli Optional per ritornare il vincitore e perdente della partita (è stato usato Optional in quanto in caso di pareggio non ci sono vincitori né perdenti)
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/b00d1ffd3078bf0b1907295f01402b6f302c376a/src/main/java/giocoscudetto/model/impl/match/MatchImpl.java#L140-L161>
2. Utilizzo i Stream+String Manipulation per il controllo della completezza delle informazioni dei club
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/f115e0761183969e263549aaade01f84c9c339c0/src/main/java/giocoscudetto/controller/impl/CreateUpdateControllerImpl.java#L35-L42>
3. Utilizzo di Stream+Lambda+Reference Method per la creazione delle squadre
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/f115e0761183969e263549aaade01f84c9c339c0/src/main/java/giocoscudetto/view/impl/initialize/ClubPanel.java#L163-L168>
4. Utilizzo di Stream+Method Reference per l'aggiornamento della classifica
<https://github.com/NicolasColi05/Gioco-Scudetto/blob/f115e0761183969e263549aaade01f84c9c339c0/src/main/java/giocoscudetto/model/impl/TableModel.java#L31-L36>

Per lo sviluppo di alcune funzionalità specifiche del progetto legate alla parte della view, ci siamo avvalsi del supporto di codici e IA online, i quali sono stati analizzati, compresi e opportunamente adattati per integrarsi con l'architettura generale del nostro gioco.

Di seguito vengono dettagliate le parti interessate:

- **Animazione del Lancio del Dado:** Per implementare l'effetto visivo del dado che viene lanciato con lo scorrere dei numeri, abbiamo utilizzato la classe `javax.swing.Timer`. La struttura del ciclo di esecuzione del timer e l'attivazione dell'evento allo scadere del tempo prendono spunto da esempi di programmazione asincrona in Swing trovati in rete.

- **Selezione dei Colori delle Squadre:** L'integrazione del Color Picker per permettere all'utente di personalizzare la propria squadra che si basa sull'utilizzo di quattro JButton a cui vengono associati un colore ciascuno.

Tutte le componenti sopra elencate non costituiscono un'acquisizione passiva di codice preesistente: ogni frammento è stato riscritto e integrato manualmente nel pieno rispetto dell'autonomia dello sviluppo del progetto.

4. Commenti Finali

4.1 Autovalutazione e lavori futuri

4.1.1 Cecchini

Il mio compito all'interno del gruppo è stato quello di sviluppare l'entità Fixtures, l'entità Match, caselle legate all'inizio, fine e sospensione della partita, la creazione di una view che permette di vedere calendario e classifica prima di ogni partita.

Sono soddisfatto della scelta di utilizzare AbstractTableModel per la visualizzazione di classifica e calendario. L'adozione del pattern Template Method ha permesso di separare correttamente la logica dei dati dalla loro rappresentazione, rispettando il pattern MVC e rendendo il codice manutenibile e facilmente aggiornabile. Ritengo che questa sia la parte più solida del mio lavoro dal punto di vista del design.

Dall'altro lato alcune parti del codice risultano ancora troppo legate a dettagli implementativi, con logica che avrebbe potuto essere ulteriormente astratta. In alcuni casi ho favorito una soluzione funzionante rispetto a una più elegante, accumulando del debito tecnico che andrebbe affrontato in una revisione futura. Il lavoro di gruppo è stato un ottimo terreno di crescita: gestire idee diverse e contrastanti non è stato sempre semplice, ma la nostra capacità di mediazione ci ha permesso di trovare ogni volta una soluzione per il progetto.

Se il progetto venisse portato avanti, l'obiettivo che mi piacerebbe sviluppare sarebbe estendere il supporto ad altri sport. L'architettura attuale lo rende fattibile senza stravolgere il design esistente: le interfacce Match, Fixtures e Club sono sufficientemente generiche da poter essere implementate per sport diversi, e i TableModel si adatterebbero facilmente a strutture dati diverse mantenendo la stessa logica di visualizzazione.

4.1.2 Colinucci

Il mio ruolo è stato idealizzare la parte della ScoreBoard, Board, GoalNet, e anche una parte sostanziale delle caselle, la cosa che sicuramente potevo apprezzare meglio sono state le implementazioni delle mie caselle (CornerBox, FreekickBox, Resultbox, PenaltyBox) avrei dovuto pensare sin da subito alla soluzione finale fatta cioè che essendo poi collegate strettamente alle interazioni con l'utente ho messo che settano delle modalità all'interno del Match, perciò il fatto di non averla studiata bene in principio mi ha portato ad avere molte difficoltà nelle implementazioni anche cambiando spesso idea.

A parte questa parentesi pietosa sono abbastanza soddisfatto del mio lavoro e anche delle classi BoardPanel, EventPanel, DicePanel, MatchPanel che tutto sommato risultano abbastanza carine da vedere nell'insieme e rispecchia molto il vero tabellone del gioco da tavolo poi sono ben accette critiche anche perchè prima d'ora le componenti grafiche non le avevo mai usate (come Graphics per esempio) e anche mai "animato delle pedine", per il resto il codice mi è sembrato ben fatto e l'utilizzo di alcuni pattern lo ha reso anche molto più comprensibile e più pulito.

Sicuramente è stata un'esperienza che nel bene e nel male mi ha insegnato molto sia nella programmazione che nell'imparare a lavorare in gruppo visto che prima di questo progetto non c'erano state occasioni per farlo.

4.1.3 Pasquinelli

Il mio compito all'interno del gruppo è stato quello di sviluppare delle classi per la gestione dei turni e per la gestione delle pedine, l'implementazione di caselle speciali inerenti a "salta un turno", "unione" e "torna all'inizio" e la creazione di una view che permette di vedere la classifica finale e il vincitore al termine di tutte le partite disputate. Personalmente il punto di forza è stato il rispetto dei pattern MVC, proprio come presenta l'EndGameView, mentre il punto in cui ho avuto più carenza è stata l'impostazione di alcuni test per verificare il corretto funzionamento delle classi e caselle. In caso si portasse avanti il progetto io continuerei in questa direzione ampliando le funzionalità del gioco, come ad esempio l'utilizzo di nomi di calciatori reali a cui attribuire ogni gol, così da poter aggiungere nella schermata finale anche la possibilità di avere una classifica marcatori.

4.1.4 Poli

Sono molto contento del risultato ottenuto da questo progetto, specialmente per come abbiamo affrontato insieme le varie problematiche riscontrate durante la fase di progettazione e implementazione delle varie classi.

Nel gruppo mi sono occupato della creazione ed aggiornamento delle informazioni dei Club e della Table: partendo dalla view ClubImpl che permette all'utente di scegliere il numero di partecipanti e i dettagli delle squadre ai due controller MatchController e CreateUpdateController che ne permettono l'aggiornamento di partita in partita.

Personalmente ritengo che un punto di forza del mio codice, seppur banale, è la coerenza che persiste tra i metodi delle mie classi, favorendo un massimo riuso del codice. In più la responsività della mia view rende sfruttabile il gioco da praticamente qualunque schermo.

Invece come punto di debolezza è il mancato uso di design pattern particolari, che tuttavia ritengo sarebbe stato difficile usare nelle mie classi (posso sbagliarmi).

Riguardo a successivi sviluppi del codice, sarebbe bello aggiungere un sistema di salvataggio dei campionati, in modo da avere uno storico consultabile a piacere e aggiungere un versione di scontro con i bot.

Appendice A

Guida utente

A1. Avvio del Gioco

All'avvio dell'applicazione viene mostrata la schermata iniziale. Per iniziare una nuova partita è necessario premere il tasto "PLAY".



1. Digitare il nome nell'apposito campo di testo. I nomi scelti non possono essere uguali e possono contenere un massimo di 9 caratteri.
2. Scegliere il colore della pedina, che rappresenterà la squadra sul tabellone.
3. Confermare la scelta premendo sul tasto "CONTINUE", che apparirà una volta completati i passaggi precedenti.
4. Se per qualsiasi motivo si vuole tornare indietro è necessario premere il tasto "BACK". In questo caso si tornerà alla schermata iniziale.

GIOCO DELLO SCUDETTO

Choose the number of player **2**

CLUB'S NAME

INTER

ROMA

PAWN'S COLOR

BACK CONTINUE

Una volta cliccato "CONTINUE" viene creato un calendario ad hoc e una classifica, che verrà aggiornata di partita in partita nel seguente modo:

- la vittoria da 3 punti, il pareggio 1 punto e la sconfitta 0 punti
- in caso di parità di punti, interviene la differenza reti, aggiornata di partita in partita facendo la differenza tra reti segnate e reti subite
- l'ordine in cui vengono mostrate le squadre corrisponde alla loro posizione in classifica

GIOCO DELLO SCUDETTO

FIXTURE		STANDINGS		
Match	Score	Club	Points	Net Difference
Roma - Inter	VS	Roma	0	0
Inter - Roma	VS	Inter	0	0

BACK CONTINUE

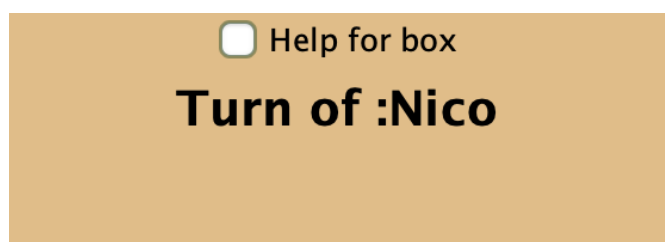
A3. Svolgimento della partita

All'inizio della partita viene determinato casualmente quale squadra inizierà. Durante il proprio turno il giocatore per lanciare i dadi deve premere il pulsante "Roll Dice" e attendere lo spostamento automatico della propria pedine sul tabellone.



I dadi usati per procedere, durante il primo tempo (prime 16 caselle), saranno due e vanno da 0 a 6 perciò in un turno si potrà stare pure fermi. I dadi non sono regolari e hanno dei pesi specifici, le probabilità di un dado sono le seguenti: 0 ha il 6,25% di uscita e gli altri numeri il 15,6%.

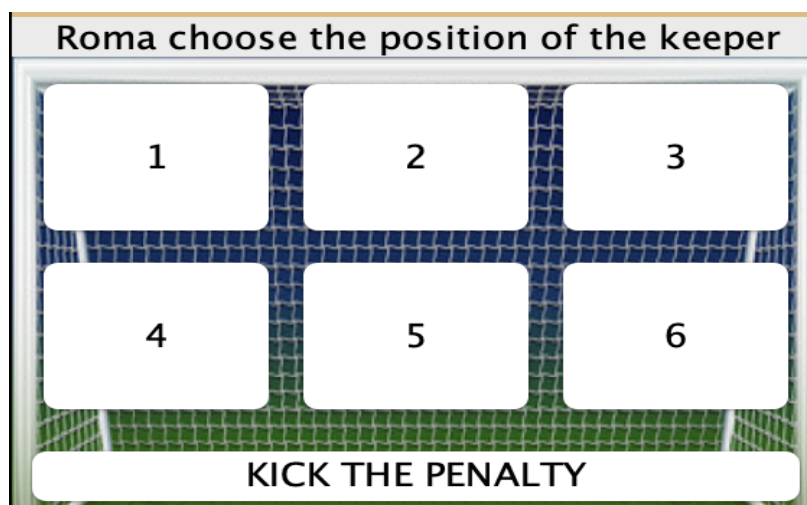
Si consiglia per chi non conosca il funzionamento di tutte le caselle di selezionare il flag Help for Box il quale ogni volta in cui salti su una casella ti spiega il suo funzionamento prima che si attivi.



All'interno del tabellone oltre normali caselle di stazionamento, di sfondo giallo, si trovano caselle speciali. Queste caselle attivano eventi particolari. In alto a destra è possibile attivare l'opzione di aiuto, utile in questi casi: se questa opzione è attiva, verrà mostrata una finestra descrittiva dell'evento.

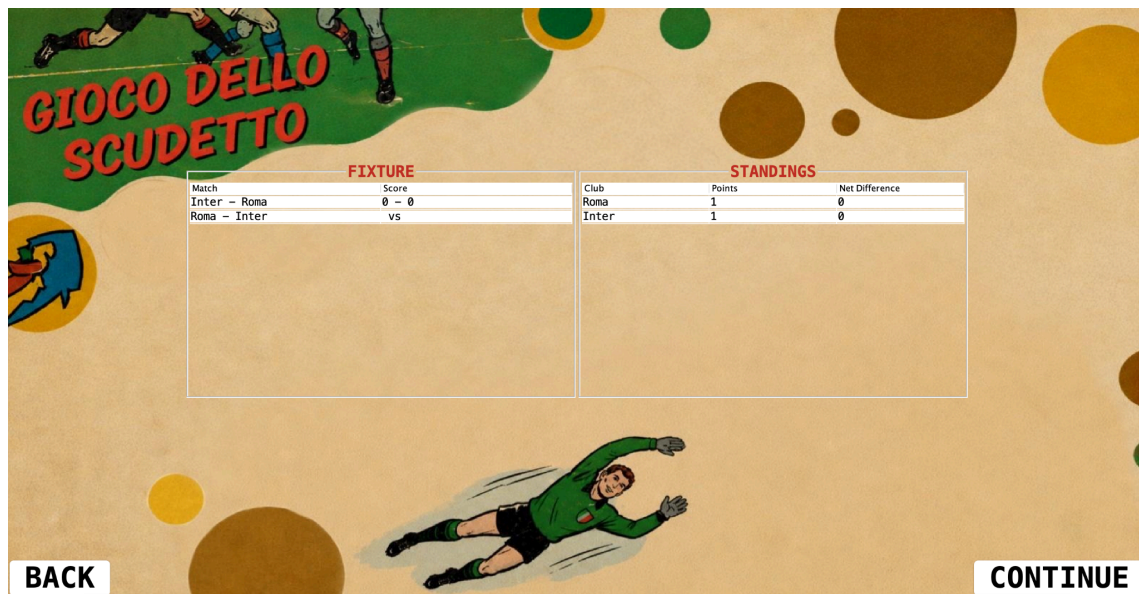
Tra queste caselle speciali si trovano:

- **RISULTATO:** premendo sul tasto “New Result” viene attribuito alla partita un nuovo risultato. Per far apparire il nuovo risultato sul tabellone e proseguire con la partita premere “Continue”;
- **PUNIZIONE:** il giocatore corrente tira i dadi. Se il risultato è 7, allora quel giocatore segna un gol. Per far apparire il nuovo risultato sul tabellone e proseguire con la partita premere “Continue”;
- **TORNA AL VIA:** il giocatore corrente che finisce sopra questa casella torna direttamente al via;
- **CALCIO D'ANGOLO:** il giocatore corrente tira i dadi premendo su “kick the corner”. Se uno dei due dadi si ferma sul numero 1 quel giocatore segna un gol. Per far apparire il nuovo risultato sul tabellone e proseguire con la partita premere “Continue”;
- **PARTITA SOSPESA:** entrambi i giocatori tornano direttamente all'inizio e la partita ricomincia da 0-0;
- **FINE PRIMO TEMPO:** per il resto della partita si usa un solo dado;
- **UNIONE:** il giocatore corrente viene raggiunto dall'altro giocatore su questa casella ed entrambi i giocatori riprendono la partita da quella posizione;
- **CALCIO DI RIGORE:** viene attribuito un rigore al giocatore corrente. Viene attivata una porta dove si trovano sei riquadri numerati che corrispondono a sei zone della porta in cui il portiere potrebbe essersi buttato. Trovare il portiere significa sbagliare il rigore. Scegliere due riquadri adiacenti che si preferiscono e premere il tasto “KICK THE PENALTY”, sapendo che ogni riquadro ha la stessa probabilità di trovare il portiere. In caso di gol fatto compare la scritta “GOAL”, se invece il portiere riesce a parare il rigore compare la scritta “NO GOAL”;



- SALTA UN TURNO: il giocatore corrente deve stare fermo un turno. L'avversario, quindi avrà a disposizione due tiri di seguito. Dopodichè riprende il giro standard;
- GOAL SUBITO: il giocatore corrente subisce un gol;
- GOAL ANNULLATO: se il giocatore corrente ha segnato uno o più gol nel corso della partita, gliene viene annullato uno;
- ZONA CESARINI: il giocatore corrente segna un gol;
- INIZIO PARTITA/FINE PARTITA: la partita inizia e termina da questa casella.

Quando il primo dei giocatori arriva sulla casella “FINE PARTITA”, il giocatore avversario lo raggiunge. Premere “CONTINUE” per la schermata successiva dove si trovano scontri e classifica parziale.



Premere nuovamente “CONTINUE” per avviare la partita successiva e attendere che le pedine tornino alla posizione di inizio partita.

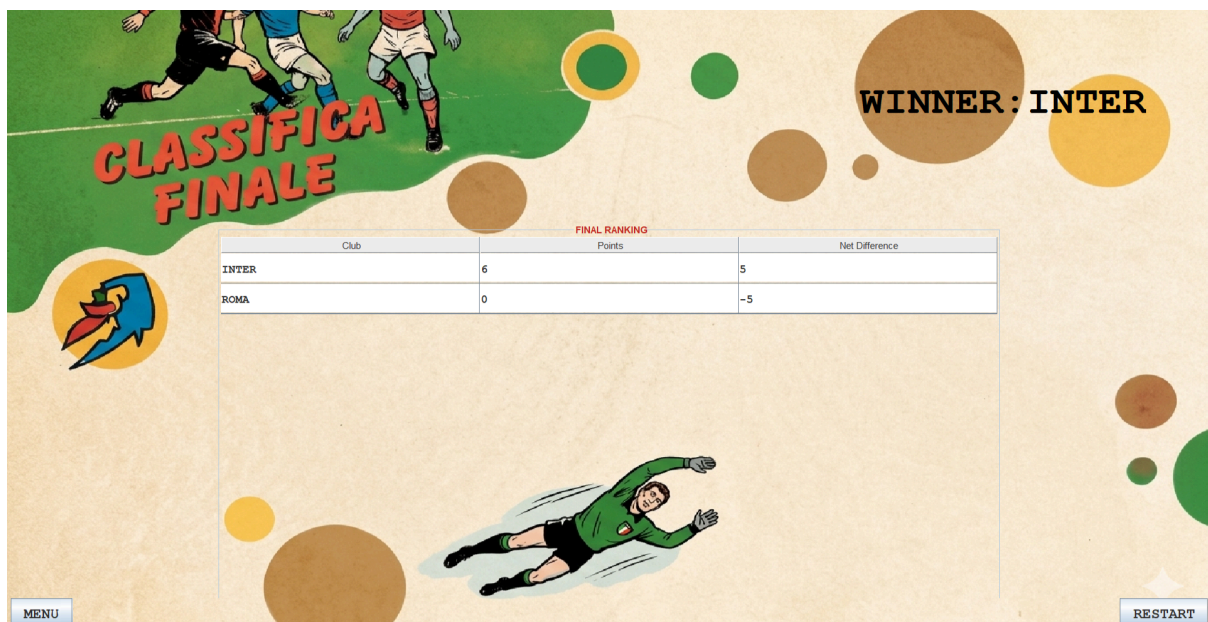
A4. Fine della partita

Quando tutte le partite previste dal campionato sono state disputate il gioco mostra la schermata finale.

In questa schermata vengono visualizzati la classifica finale, ordinata in base ai punti ottenuti e alla differenza reti, e il club vincitore del campionato, in alto a destra.

Sono poi disponibili due tasti:

- il tasto "MENU" riporta alla schermata di configurazione giocatori
- il tasto "RESTART" riavvia il precedente campionato, con gli stessi giocatori e gli stessi club.



Esercitazioni di laboratorio

B.0.1 gianluca.poli8@studio.unibo.it

Laboratorio 6: <https://virtuale.unibo.it/mod/forum/discuss.php?d=206731#p284105>

Laboratorio 7: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284996>

Laboratorio 8: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286180>

Laboratorio 9: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p286584>

Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288215>

Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289530>

Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290528>

B.0.2 nicolas.colinucci@studio.unibo.it

Laboratorio 07 : <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p285058>

Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286163>

Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287123>

Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288469>

Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289593>

Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290705>

B.0.3 federico.cecchini7@studio.unibo.it

Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286209>

B.0.4 Diego.pasquinelli2@studio.unibo.it